

OpenMetadata 离散制造业物料主数据治理：技术实施详细指南

文档说明

本文档面向制造业 IT 团队和数据治理项目组，提供基于 OpenMetadata 的物料主数据治理项目从规划到落地的完整技术实施路径。包含具体的工作步骤、配置文件示例、SQL 代码、命令行操作和项目管理建议。

适用范围：拥有 SAP ERP (ECC/S4)、金蝶 K3/K3 Cloud、MES、WMS、PLM 等多系统环境的离散制造企业。

OpenMetadata 版本：1.12.x (截至 2026 年 5 月)

第一部分：项目准备与规划

1.1 项目启动清单

- 成立项目团队 (建议配置)
 - └ 项目经理 × 1 (数据治理经验)
 - └ 数据架构师 × 1 (熟悉制造业数据模型)
 - └ SAP 顾问 × 1 (熟悉 MARA/MAKT/MARC 等物料表)
 - └ 数据库工程师 × 1 (MySQL/PostgreSQL/SQL Server)
 - └ 业务代表 × 2 (采购 + 生产/质量各 1 人)
 - └ OpenMetadata 开发工程师 × 1-2
- 确定项目范围
 - └ 治理对象：物料主数据 (本期) / 供应商主数据 (下期)
 - └ 系统范围：SAP + 金蝶 + MES (本期) / WMS + PLM (下期)
 - └ 数据范围：新增物料 (本期) / 存量物料清洗 (下期)
- 硬件资源准备
 - └ 服务器：4C8G 起步 (测试), 8C16G (生产)
 - └ 存储：100GB SSD (起步, 视数据量增长)
 - └ 网络：可访问各业务系统数据库 (需开放端口)
- 环境准备
 - └ 测试环境 (Dev): 用于开发和验证
 - └ 预发环境 (Staging): 模拟生产环境
 - └ 生产环境 (Prod): 正式运行

1.2 项目里程碑计划 (12 周)

阶段	周期	里程碑	交付物
Phase 1: 基础部署	Week 1-2	OpenMetadata 平台部署完成	可访问的治理平台、连接测试通过
Phase 2: 元数据采集	Week 3-4	全系统元数据采集完成	完整的物料数据目录、数据画像报告
Phase 3: 质量规则	Week 5-7	数据质量测试体系上线	20+ 质量测试用例、质量报告
Phase 4: 血缘建设	Week 8-9	列级血缘链路打通	全链路血缘图谱、影响分析报告
Phase 5: 治理运营	Week 10-11	治理流程正式运行	Glossary 术语表、KPI 看板、告警通知

Table 1 – continued

阶段	周期	里程碑	交付物
Phase 6：验收优化	Week 12	项目验收	验收报告、运维手册、培训材料

第二部分：OpenMetadata 平台部署

2.1 部署方式选择

方式	适用场景	复杂度	维护成本
Docker Compose	测试环境、中小型企业 (<100 万物料)	低	低
Kubernetes (Helm)	大型企业、高可用要求	中	中
Bare Metal	有专职运维团队、性能要求 极高	高	高

推荐：测试环境用 Docker Compose，生产环境用 Kubernetes。

2.2 Docker Compose 部署（测试/开发环境）

Step 1: 环境准备

```
# 系统要求
# - Linux: CentOS 7+/Ubuntu 18.04+/RHEL 8+
# - Docker: 20.10+
# - Docker Compose: 2.x+
# - 内存: >= 8GB
# - 磁盘: >= 50GB

# 检查 Docker 版本
docker --version
docker compose version

# 创建项目目录
mkdir -p /opt/openmetadata
cd /opt/openmetadata
```

Step 2: 下载官方部署包

```
# 下载最新 1.12.x 版本
export OM_VERSION=1.12.6
wget https://github.com/open-metadata/OpenMetadata/releases/download/${OM_VERSION}-release/
↩ docker-compose.tar.gz

# 解压
tar -xzf docker-compose.tar.gz
cd docker-compose
```

```
# 目录结构说明
# .
# └─ docker-compose.yml          # 主编排文件
# └─ docker-compose-postgres.yml # PostgreSQL 版本（推荐）
# └─ docker-compose-mysql.yml    # MySQL 版本
# └─ env-mysql                   # MySQL 环境变量
# └─ env-postgres                # PostgreSQL 环境变量
# └─ ingestion/                  # 数据采配置
```

Step 3: 配置环境变量

```
# 编辑 PostgreSQL 环境变量文件
cat > env-postgres << 'EOF'
# OpenMetadata 服务端配置
AUTHORIZER_CLASS_NAME=org.openmetadata.service.security.DefaultAuthorizer
AUTHORIZER_REQUEST_FILTER=org.openmetadata.service.security.JwtFilter
AUTHORIZER_ADMIN_PRINCIPALS=[admin]
AUTHORIZER_ALLOWED_DOMAINS=[]
AUTHORIZER_INGESTION_PRINCIPALS=[ingestion-bot]
AUTHORIZER_PRINCIPAL_DOMAIN=open-metadata.org

# JWT 配置（生产环境请修改密钥）
RSA_PUBLIC_KEY_FILE_PATH=/openmetadata/jwt/public_key.der
RSA_PRIVATE_KEY_FILE_PATH=/openmetadata/jwt/private_key.der
JWT_ISSUER=open-metadata.org
JWT_KEY_ID=Gb389a-9f76-gdjs-a92j-0242bk94356

# 数据库配置（PostgreSQL）
DB_DRIVER_CLASS=org.postgresql.Driver
DB_SCHEME=jdbc:postgresql
DB_PARAMS=?allowPublicKeyRetrieval=true&useSSL=false&serverTimezone=UTC
DB_USER=postgres
DB_PASSWORD=postgres_password_2026      # 生产环境请修改！
DB_HOST=postgres
DB_PORT=5432
DB_DATABASE=openmetadata_db

# Elasticsearch 配置
SEARCH_TYPE=elasticsearch
ELASTICSEARCH_USER=
ELASTICSEARCH_PASSWORD=
ELASTICSEARCH_SOCKET_TIMEOUTSecs=60
ELASTICSEARCH_CLUSTER_ALIAS=
ELASTICSEARCH_TRUST_STORE_PATH=
ELASTICSEARCH_TRUST_STORE_PASSWORD=
ELASTICSEARCH_CONNECTION_TIMEOUT_SECS=5
ELASTICSEARCH_PORT=9200
ELASTICSEARCH_SCHEME=http
ELASTICSEARCH_HOST=elasticsearch

# 服务器配置
```

```
SERVER_HOST_API_URL=http://localhost:8585/api
SERVER_ADMIN_PORT=8586
EOF
```

Step 4: 启动服务

```
# 启动 PostgreSQL 版本 (推荐)
docker compose -f docker-compose-postgres.yml up -d

# 查看服务状态
docker compose -f docker-compose-postgres.yml ps

# 预期输出
# NAME                STATUS              PORTS
# openmetadata_server Up 30 seconds      0.0.0.0:8585->8585/tcp
# openmetadata_ingestion Up 30 seconds
# elasticsearch        Up 30 seconds      0.0.0.0:9200->9200/tcp
# postgres             Up 30 seconds      0.0.0.0:5432->5432/tcp

# 查看日志
docker logs -f openmetadata_server

# 等待启动完成 (看到 "Started OpenMetadata in xxx seconds")
```

Step 5: 验证部署

```
# 检查 API 健康状态
curl http://localhost:8585/api/v1/health-check

# 预期返回
# {"status":"healthy","version":"1.12.6"}

# 浏览器访问
# http://localhost:8585
# 默认账号: admin@open-metadata.org / admin
```

2.3 Kubernetes 部署 (生产环境)

Step 1: 安装 Helm Chart

```
# 添加 OpenMetadata Helm 仓库
helm repo add open-metadata https://helm.open-metadata.io
helm repo update

# 创建命名空间
kubectl create namespace openmetadata

# 安装 (使用默认配置)
helm install openmetadata open-metadata/openmetadata \
  --namespace openmetadata \
  --set openmetadata.config.database.host=postgres.openmetadata.svc.cluster.local \
  --set openmetadata.config.database.passwordSecretRef=db-password-secret \
```

```
--set openmetadata.config.elasticsearch.host=elasticsearch.openmetadata.svc.cluster.local
```

```
# 查看 Pod 状态
```

```
kubectl get pods -n openmetadata
```

Step 2: 配置高可用（生产必需）

```
# values-production.yaml
openmetadata:
  config:
    # 数据库使用外部托管 PostgreSQL
    database:
      host: manufacturing-db.company.internal
      port: 5432
      databaseName: openmetadata_prod
      auth:
        password:
          secretRef: om-db-secret
          secretKey: password

    # Elasticsearch 使用外部集群
    elasticsearch:
      host: es-cluster.company.internal
      port: 9200
      scheme: https
      auth:
        username: openmetadata
        password:
          secretRef: om-es-secret
          secretKey: password

    # JWT 密钥（使用 Kubernetes Secret）
    authorizer:
      className: org.openmetadata.service.security.DefaultAuthorizer
      containerRequestFilter: org.openmetadata.service.security.JwtFilter
      adminPrincipals:
        - admin
      principalDomain: open-metadata.org

  # 资源限制
  resources:
    requests:
      memory: "4Gi"
      cpu: "2000m"
    limits:
      memory: "8Gi"
      cpu: "4000m"

  # 副本数
  replicaCount: 2
```

```
# 持久化存储
persistence:
  enabled: true
  storageClass: "fast-ssd"
  size: 50Gi

# Airflow (数据采集调度)
airflow:
  enabled: true
  airflow:
    image:
      repository: docker.getcollate.io/openmetadata/ingestion
      tag: 1.12.6
    executor: CeleryExecutor
  workers:
    replicas: 2
    resources:
      requests:
        memory: "2Gi"
        cpu: "1000m"
```

```
# 使用生产配置部署
helm install openmetadata open-metadata/openmetadata \
  --namespace openmetadata \
  -f values-production.yaml
```

2.4 安全配置（生产环境必做）

Step 1: 修改默认密码

```
# 登录后, 进入 Settings > Users > admin
# 修改默认密码为强密码 (至少 12 位, 包含大小写 + 数字 + 特殊字符)
```

Step 2: 配置 SSO（单点登录）

```
# 以 LDAP/Active Directory 为例
# 进入 Settings > Configuration > SSO

# 配置参数
cat > sso-ldap-config.json << 'EOF'
{
  "provider": "ldap",
  "authority": "ldap://ad.company.internal:389",
  "clientId": "openmetadata",
  "callbackUrl": "http://om.company.internal/callback",
  "scopes": ["openid", "profile", "email"],
  "ldapConfiguration": {
    "host": "ad.company.internal",
    "port": 389,
    "dnAdminPrincipal": "CN=om-service,OU=Services,DC=company,DC=internal",
    "dnAdminPassword": "${LDAP_PASSWORD}",
    "userBaseDN": "OU=Users,DC=company,DC=internal",
```

```
"groupBaseDN": "OU=Groups,DC=company,DC=internal",
"roleAttributeName": "memberOf",
"mailAttributeName": "mail",
"maxPoolSize": 3
}
}
EOF
```

Step 3: 配置 RBAC（基于角色的访问控制）

```
# 建议的制造业角色体系
# 1. DataAdmin（数据管理员）：全权限，IT 团队
# 2. DataSteward（数据管家）：可编辑 Glossary、审批质量测试，业务专家
# 3. DataAnalyst（数据分析师）：可查看元数据和血缘，不可修改
# 4. DataConsumer（数据消费者）：只读权限，普通业务人员

# 创建团队
curl -X POST http://localhost:8585/api/v1/teams \
  -H "Content-Type: application/json" \
  -u admin:admin \
  -d '{
    "name": " 物料数据治理组",
    "description": " 负责物料主数据的治理和质量管理",
    "teamType": "Group",
    "parents": []
  }'
```

第三部分：连接器配置与元数据采集

3.1 SAP ERP 连接器配置

Step 1: SAP 端准备工作

```
-- 在 SAP 中创建只读用户（ BASIS 团队执行 ）
-- 事务码：SU01
-- 用户名：OPENMETADATA
-- 角色：只读访问物料主数据表

-- 需要访问的核心表
-- MARA   - 一般物料数据
-- MAKT   - 物料描述
-- MARC   - 工厂级别的物料数据
-- MARM   - 物料计量单位
-- T023   - 物料组
-- T134   - 物料类型
-- T006   - 计量单位文本

-- 创建 SAP RFC 连接（如果使用 RFC 连接器）
-- 事务码：SM59
-- 连接类型：TCP/IP
```

```
-- 程序 ID: OPENMETADATA_RFC
```

Step 2: OpenMetadata 中配置 SAP HANA/SAP ERP 连接器

```
# 方式一: 通过 SAP HANA 数据库直连 (推荐, 最简单)
# 前提: SAP 数据已同步到 HANA

# 进入 OpenMetadata UI
# 1. 导航到 Settings > Services > Database Services
# 2. 点击 "Add New Service"
# 3. 选择 "SAP HANA"

# 填写连接信息
cat > sap-hana-connection.json << 'EOF'
{
  "serviceName": "SAP-HANA-PROD",
  "serviceType": "SapHANA",
  "connection": {
    "config": {
      "type": "SapHANA",
      "username": "OPENMETADATA",
      "password": "${SAP_PASSWORD}",
      "hostPort": "sap-hana.company.internal:30015",
      "database": "PROD",
      "databaseSchema": "ERP",
      "connectionOptions": {
        "encrypt": "true",
        "sslValidateCertificate": "false"
      },
      "supportedPipelineTypes": ["metadata", "usage", "lineage", "profiler"]
    }
  }
}
EOF

# 方式二: 通过 SAP ERP 连接器 (RFC 方式)
# 适用于直接从 SAP ABAP 系统抽取
cat > sap-erp-connection.json << 'EOF'
{
  "serviceName": "SAP-ERP-PROD",
  "serviceType": "SapERP",
  "connection": {
    "config": {
      "type": "SapERP",
      "hostPort": "sap-erp.company.internal:3300",
      "username": "OPENMETADATA",
      "password": "${SAP_PASSWORD}",
      "clientId": "100",
      "language": "ZH",
      "tables": [
        "MARA",
```



```
        "MAKT",
        "MARC",
        "MARM",
        "T023",
        "T134"
    ]
}
}
}
EOF
```

Step 3: 配置元数据采 Pipeline

```
# 进入 OpenMetadata UI
# 1. 导航到 Settings > Ingestion > Add Ingestion
# 2. 选择 "Metadata"

# 配置采参数
cat > sap-metadata-ingestion.json << 'EOF'
{
  "name": "sap-material-metadata",
  "serviceName": "SAP-HANA-PROD",
  "pipelineType": "metadata",
  "sourceConfig": {
    "config": {
      "type": "DatabaseMetadata",
      "markDeletedTables": true,
      "includeTables": true,
      "includeViews": true,
      "schemaFilterPattern": {
        "includes": ["ERP"]
      },
    },
    "tableFilterPattern": {
      "includes": ["MARA", "MAKT", "MARC", "MARM", "T023", "T134", "T006"]
    },
    "useFqnForFiltering": false,
    "overwriteOwner": false
  }
},
"scheduleInterval": {
  "type": "custom",
  "cron": "0 2 * * *" // 每天凌晨 2 点执行
}
}
EOF

# 立即执行采
# UI 中点击 "Run Now" 或等待定时触发
```

Step 4: 验证采结果

```
# 检查采日志
```

```
# UI 中导航到 Ingestion > 点击 Pipeline 名称 > Logs

# 预期输出
# [2026-05-01T02:00:05] INFO - Scanned schema: ERP
# [2026-05-01T02:00:10] INFO - Scanned table: MARA (125,678 rows)
# [2026-05-01T02:00:15] INFO - Scanned table: MAKT (125,678 rows)
# [2026-05-01T02:00:20] INFO - Scanned table: MARC (456,234 rows)
# [2026-05-01T02:00:25] INFO - Scanned table: MARM (234,567 rows)
# [2026-05-01T02:00:30] INFO - Metadata ingestion completed successfully
```

3.2 金蝶 K3 Cloud 连接器配置

金蝶 K3 Cloud 使用 SQL Server/MySQL 作为后端数据库，OpenMetadata 可直接通过数据库连接器访问。

Step 1: 确认数据库类型和连接信息

```
# 金蝶 K3 Cloud 常见数据库配置
# - 数据库类型: SQL Server 2016+/MySQL 5.7+
# - 默认数据库名: AIS202xxxxx (每家客户不同)
# - 物料相关表: T_BD_MATERIAL, T_BD_MATERIAL_L, T_BD_MATERIALBASE...
```

Step 2: 创建数据库用户 (DBA 执行)

```
-- SQL Server 版本
CREATE LOGIN openmetadata WITH PASSWORD = 'Kingdee_OM_2026';
CREATE USER openmetadata FOR LOGIN openmetadata;
GRANT SELECT ON T_BD_MATERIAL TO openmetadata;
GRANT SELECT ON T_BD_MATERIAL_L TO openmetadata;
GRANT SELECT ON T_BD_MATERIALBASE TO openmetadata;
GRANT SELECT ON T_BD_MATERIALSTOCK TO openmetadata;
GRANT SELECT ON T_BAS_ASSISTANTDATA_L TO openmetadata;
GRANT SELECT ON T_BAS_ASSISTANTDATAENTRY_L TO openmetadata;
GO

-- MySQL 版本
CREATE USER 'openmetadata'@'%' IDENTIFIED BY 'Kingdee_OM_2026';
GRANT SELECT ON AIS202xxxxx.T_BD_MATERIAL TO 'openmetadata'@'%';
GRANT SELECT ON AIS202xxxxx.T_BD_MATERIAL_L TO 'openmetadata'@'%';
GRANT SELECT ON AIS202xxxxx.T_BD_MATERIALBASE TO 'openmetadata'@'%';
FLUSH PRIVILEGES;
```

Step 3: OpenMetadata 中配置连接器

```
# SQL Server 版本
cat > kingdee-mssql-connection.json << 'EOF'
{
  "serviceName": "Kingdee-K3Cloud-PROD",
  "serviceType": "Mssql",
  "connection": {
    "config": {
      "type": "Mssql",
      "username": "openmetadata",
```

```

        "password": "${KINGDEE_PASSWORD}",
        "hostPort": "kingdee-db.company.internal:1433",
        "database": "AIS202xxxxx",
        "driver": "ODBC Driver 18 for SQL Server",
        "connectionOptions": {
            "TrustServerCertificate": "yes"
        }
    }
}
}
EOF

# MySQL 版本
cat > kingdee-mysql-connection.json << 'EOF'
{
    "serviceName": "Kingdee-K3Cloud-PROD",
    "serviceType": "Mysql",
    "connection": {
        "config": {
            "type": "Mysql",
            "username": "openmetadata",
            "password": "${KINGDEE_PASSWORD}",
            "hostPort": "kingdee-mysql.company.internal:3306",
            "databaseSchema": "AIS202xxxxx",
            "connectionOptions": {
                "useSSL": "false",
                "serverTimezone": "Asia/Shanghai"
            }
        }
    }
}
EOF

```

Step 4: 配置采 Pipeline

```

cat > kingdee-metadata-ingestion.json << 'EOF'
{
    "name": "kingdee-material-metadata",
    "serviceName": "Kingdee-K3Cloud-PROD",
    "pipelineType": "metadata",
    "sourceConfig": {
        "config": {
            "type": "DatabaseMetadata",
            "markDeletedTables": true,
            "tableFilterPattern": {
                "includes": [
                    "T_BD_MATERIAL",
                    "T_BD_MATERIAL_L",
                    "T_BD_MATERIALBASE",
                    "T_BD_MATERIALSTOCK",
                    "T_BD_MATERIALSALE",

```

```

        "T_BD_MATERIALPURCHASE",
        "T_BD_MATERIALPLAN",
        "T_BD_MATERIALPRODUCE",
        "T_BAS_ASSISTANTDATA_L",
        "T_BAS_ASSISTANTDATAENTRY_L"
    ]
}
},
"scheduleInterval": {
    "type": "custom",
    "cron": "0 3 * * *"    // 每天凌晨 3 点执行（错开 SAP）
}
}
EOF

```

3.3 MES/WMS/PLM 连接器配置

MES 系统（以常见的 SQL Server/MySQL 后端为例）

```

# MES 数据库连接器
cat > mes-connection.json << 'EOF'
{
    "serviceName": "MES-PROD",
    "serviceType": "Mysql",
    "connection": {
        "config": {
            "type": "Mysql",
            "username": "openmetadata",
            "password": "${MES_PASSWORD}",
            "hostPort": "mes-db.company.internal:3306",
            "databaseSchema": "mes_prod",
            "tableFilterPattern": {
                "includes": [
                    "material_master",
                    "material_bom",
                    "material_routing",
                    "work_order_material"
                ]
            }
        }
    }
}
EOF

```

WMS 系统

```

cat > wms-connection.json << 'EOF'
{
    "serviceName": "WMS-PROD",
    "serviceType": "Postgres",
    "connection": {

```

```

    "config": {
      "type": "Postgres",
      "username": "openmetadata",
      "password": "${WMS_PASSWORD}",
      "hostPort": "wms-db.company.internal:5432",
      "database": "wms_prod",
      "tableFilterPattern": {
        "includes": [
          "sku_master",
          "inventory_item",
          "receiving_material"
        ]
      }
    }
  }
}
EOF

```

PLM 系统

```

cat > plm-connection.json << 'EOF'
{
  "serviceName": "PLM-PROD",
  "serviceType": "Oracle",
  "connection": {
    "config": {
      "type": "Oracle",
      "username": "openmetadata",
      "password": "${PLM_PASSWORD}",
      "hostPort": "plm-db.company.internal:1521",
      "oracleServiceName": "PLMPROD",
      "tableFilterPattern": {
        "includes": [
          "PART_MASTER",
          "PART_REVISION",
          "MATERIAL_SPEC",
          "BOM_HEADER",
          "BOM_LINE"
        ]
      }
    }
  }
}
EOF

```

3.4 数据画像（Data Profiler）配置

数据画像自动分析表的数据分布，为质量测试提供基线。

```

# 配置 SAP MARA 表的画像
cat > sap-profiler.json << 'EOF'
{

```

```

"name": "sap-material-profiler",
"serviceName": "SAP-HANA-PROD",
"pipelineType": "profiler",
"sourceConfig": {
  "config": {
    "type": "Profiler",
    "generateSampleData": true,
    "sampleRows": 100,
    "profileSampleType": "PERCENTAGE",
    "profileSample": 10,
    "threadCount": 5,
    "tableConfig": [
      {
        "fullyQualifiedName": "SAP-HANA-PROD.ERP.MARA",
        "profileSample": 100,          // MARA 全量画像
        "columnConfig": [
          {
            "columnName": "MATNR",    // 物料编码
            "metrics": ["MIN", "MAX", "COUNT", "DISTINCT_COUNT", "PROPORTION"]
          },
          {
            "columnName": "MTART",    // 物料类型
            "metrics": ["DISTINCT_COUNT", "DISTINCT_PROPORTION"]
          },
          {
            "columnName": "MATKL",    // 物料组
            "metrics": ["DISTINCT_COUNT", "DISTINCT_PROPORTION"]
          }
        ]
      },
      {
        "fullyQualifiedName": "SAP-HANA-PROD.ERP.MAKT",
        "profileSample": 100
      }
    ]
  },
  "scheduleInterval": {
    "type": "custom",
    "cron": "0 4 * * 0"           // 每周日凌晨 4 点执行
  }
}
EOF

```

第四部分：数据质量测试体系构建

4.1 制造业物料数据质量测试矩阵

测试类别	测试名称	适用表	优先级	测试类型
完整性	物料编码非空	MARA,	P0	Column Values to be Not Null
完整性	物料描述非空	MAKT,	P0	Column Values to be Not Null
完整性	物料类型非空	MARA,	P0	Column Values to be Not Null
完整性	必填字段填充率	T_BD_MATERIALBASE	P0	Column Values to Match Regex
规范性	物料编码格式	所有物料表	P0	Column Values to Match Regex
规范性	计量单位标准化	MARA	P1	Column Values to be in Set
规范性	物料类型有效性	T_BD_MATERIALBASE	P1	Column Values to be in Set
规范性	描述长度合规	MARA	P1	Column Value Lengths to be Between
一致性	重复物料检测	MAKT	P0	Custom SQL Test
一致性	一码多物检测	MARA + MAKT	P0	Custom SQL Test
一致性	跨系统单位一致	MARA	P1	Custom SQL Test
		material_master		
时效性	数据新鲜度	MARA +	P2	Table Freshness
准确性	分类体系一致性	MARA	P1	Custom SQL Test
准确性	规格型号格式	MARA + T023	P2	Column Values to Match Regex
		MAKT		

4.2 基础质量测试配置（UI 方式）

Test 1：物料编码非空检查（MARA.MATNR）

操作路径：

1. 导航到 Explore > Tables > SAP-HANA-PROD > ERP > MARA

2. 切换到 "Profiler & Data Quality" Tab

3. 点击 "Add Test"

4. 选择测试类型：Column Values to be Not Null

5. 配置参数：

- Column: MATNR

- Test Name: MARA_MATNR_NotNull

- Description: 物料编码不能为空

6. 点击 "Submit"

Test 2：物料编码格式检查（正则表达式）

测试类型: Column Values to Match Regex

配置参数：

- Column: MATNR

- Regex Pattern: `^[A-Z0-9]{6,18}$` # 6-18 位大写字母或数字

```
- Test Name: MARA_MATNR_FormatCheck
```

Test 3: 计量单位在白名单中

测试类型: Column Values to be in Set

配置参数:

- Column: MEINS
- Allowed Values: KG, G, PCS, EA, M, MM, L, ML, BOX, SET, PAIR, ROLL
- Test Name: MARA_MEINS_UnitStandard

Test 4: 描述长度合规

测试类型: Column Value Lengths to be Between

配置参数:

- Column: MAKTX
- Min Length: 5
- Max Length: 200
- Test Name: MAKT_MAKTX_LengthCheck

4.3 高级质量测试: Test Library 自定义 SQL 测试

对于制造业复杂的业务规则, 需要使用 Test Library 编写自定义 SQL。

Step 1: 创建重复物料检测测试

```
-- Test Definition: Duplicate Material Detection
-- 检测逻辑: 物料名称 + 规格型号相同但编码不同 = 疑似重复

-- 步骤 1: 在 OpenMetadata 中进入 Test Library
-- Observability > Test Library > Add Test Definition

-- 步骤 2: 填写基本信息
-- Name: duplicate_material_detection
-- Display Name: 重复物料检测
-- Description: 检测物料名称和规格相同但编码不同的记录
-- Level: Table      -- 表级测试
-- State: Active

-- 步骤 3: 编写 SQL 表达式
SELECT
    material_name,
    specification,
    COUNT(DISTINCT material_code) as code_count,
    STRING_AGG(material_code, ', ') as codes
FROM {{ table_name }}
WHERE material_name IS NOT NULL
    AND specification IS NOT NULL
GROUP BY material_name, specification
HAVING COUNT(DISTINCT material_code) > 1

-- 步骤 4: 定义参数
-- Parameter 1:
-- Name: table_name
```



```
-- Type: STRING
-- Required: Yes
-- Description: 要检测的物料表名

-- 步骤 5: 选择支持的数据源
-- SAP HANA
-- MySQL
-- PostgreSQL
-- SQL Server
-- Oracle
```

Step 2: 创建一码多物检测测试

```
-- Test Definition: One Code Multiple Materials Detection
-- 检测逻辑: 同一编码对应不同的物料名称

-- Name: one_code_multiple_materials
-- Display Name: 一码多物检测
-- Level: Table

SELECT
    material_code,
    COUNT(DISTINCT material_name) as name_count,
    STRING_AGG(DISTINCT material_name, ' | ') as names
FROM {{ table_name }}
WHERE material_code IS NOT NULL
GROUP BY material_code
HAVING COUNT(DISTINCT material_name) > 1
```

Step 3: 创建跨系统单位一致性检测

```
-- Test Definition: Cross-System Unit Consistency
-- 检测逻辑: SAP 和 MES 中同一物料的计量单位不一致

-- 注意: 此测试需要能访问两个数据库, 或使用 Federated Query

-- 对于同一数据库内的多个 schema:
SELECT
    s.material_code,
    s.unit as sap_unit,
    m.unit as mes_unit
FROM {{ sap_table }} s
JOIN {{ mes_table }} m ON s.material_code = m.material_code
WHERE s.unit != m.unit
```

Step 4: 创建分类体系一致性检测

```
-- Test Definition: Classification Consistency Check
-- 检测逻辑: 物料分类必须在预定义的分类体系中

SELECT
    material_code,
```

```

material_group
FROM {{ table_name }}
WHERE material_group NOT IN (
    SELECT group_code FROM {{ classification_table }}
)

```

Step 5: 创建物料编码连续性检测（发现编码跳跃）

```

-- Test Definition: Material Code Continuity Check
-- 检测逻辑: 发现编码序列中的大段跳跃（可能遗漏物料）

WITH code_numbers AS (
    SELECT
        CAST(SUBSTRING(material_code, 2) AS INT) as code_num
    FROM {{ table_name }}
    WHERE material_code REGEXP '^{{ prefix }}[0-9]+$'
),
code_gaps AS (
    SELECT
        code_num,
        LEAD(code_num) OVER (ORDER BY code_num) as next_code_num
    FROM code_numbers
)
SELECT
    code_num + 1 as gap_start,
    next_code_num - 1 as gap_end,
    next_code_num - code_num - 1 as gap_size
FROM code_gaps
WHERE next_code_num - code_num > {{ max_allowed_gap }}

-- Parameters:
-- table_name: STRING, Required
-- prefix: STRING, Required, Default: 'M'
-- max_allowed_gap: INT, Required, Default: 100

```

4.4 质量测试批量创建（YAML 方式）

对于大规模配置，使用 YAML 文件批量导入更高效。

```

# material_quality_tests.yaml
# 制造业物料数据质量测试批量配置

testCases:
  # ===== 完整性测试 =====
  - name: MARA_MATNR_NotNull
    entityFQN: SAP-HANA-PROD.ERP.MARA.MATNR
    testDefinition: columnValuesToBeNotNull
    parameterValues: []

  - name: MARA_MTART_NotNull
    entityFQN: SAP-HANA-PROD.ERP.MARA.MTART
    testDefinition: columnValuesToBeNotNull

```

```

parameterValues: []

- name: MARA_MEINS_NotNull
  entityFQN: SAP-HANA-PROD.ERP.MARA.MEINS
  testDefinition: columnValuesToBeNotNull
  parameterValues: []

- name: MAKT_MAKTX_NotNull
  entityFQN: SAP-HANA-PROD.ERP.MAKT.MAKTX
  testDefinition: columnValuesToBeNotNull
  parameterValues: []

# ===== 规范性测试 =====

- name: MARA_MATNR_FormatCheck
  entityFQN: SAP-HANA-PROD.ERP.MARA.MATNR
  testDefinition: columnValuesToMatchRegex
  parameterValues:
    - name: regex
      value: "[A-Z0-9]{6,18}$"

- name: MARA_MEINS_UnitSet
  entityFQN: SAP-HANA-PROD.ERP.MARA.MEINS
  testDefinition: columnValuesToBeInSet
  parameterValues:
    - name: allowedValues
      value: "['KG', 'G', 'PCS', 'EA', 'M', 'MM', 'L', 'ML', 'BOX', 'SET', 'PAIR', 'ROLL']"

- name: MARA_MTART_TypeSet
  entityFQN: SAP-HANA-PROD.ERP.MARA.MTART
  testDefinition: columnValuesToBeInSet
  parameterValues:
    - name: allowedValues
      value: "['ROH', 'HALB', 'FERT', 'HAWA', 'HIBE', 'ERSA', 'UNBW']"
      # ROH= 原材料, HALB= 半成品, FERT= 成品, HAWA= 贸易货物, HIBE= 辅料, ERS= 备件, UNBW= 非估价物料

- name: MAKT_MAKTX_LengthCheck
  entityFQN: SAP-HANA-PROD.ERP.MAKT.MAKTX
  testDefinition: columnValueLengthsToBeBetween
  parameterValues:
    - name: minLength
      value: "5"
    - name: maxLength
      value: "200"

# ===== 自定义 SQL 测试（需要在 Test Library 中预先定义） =====

- name: MARA_DuplicateMaterialCheck
  entityFQN: SAP-HANA-PROD.ERP.MARA
  testDefinition: duplicate_material_detection # 自定义测试定义
  parameterValues:
    - name: table_name
      value: "MARA"

```

```

- name: MARA_OneCodeMultiMaterialCheck
  entityFQN: SAP-HANA-PROD.ERP.MARA
  testDefinition: one_code_multiple_materials      # 自定义测试定义
  parameterValues: []

# ===== 金蝶系统测试 =====
- name: KD_TBDMATERIAL_FMaterialID_NotNull
  entityFQN: Kingdee-K3Cloud-PROD.AIS202xxxxx.T_BD_MATERIAL.FMaterialID
  testDefinition: columnValuesToBeNotNull
  parameterValues: []

- name: KD_TBDMATERIAL_FNumber_FormatCheck
  entityFQN: Kingdee-K3Cloud-PROD.AIS202xxxxx.T_BD_MATERIAL.FNumber
  testDefinition: columnValuesToMatchRegex
  parameterValues:
    - name: regex
      value: "^\\d{3}\\.\\d{2}\\.\\d{4}$"      # 金蝶典型编码格式: XXX.XX.XXXX

# 使用 OpenMetadata Python SDK 批量导入
# 见下文 SDK 示例

```

4.5 使用 Python SDK 自动化配置

```

#!/usr/bin/env python3
"""
OpenMetadata 制造业物料数据质量测试批量配置脚本
需要安装: pip install openmetadata-ingestion[sdk]
"""

import yaml
from metadata.generated.schema.entity.data.table import Table
from metadata.generated.schema.tests.basic import (
    BasicExecutionConfig,
    TestCaseResult,
    TestCaseStatus,
    TestResultValue,
)
from metadata.generated.schema.tests.testCase import TestCase
from metadata.generated.schema.tests.testDefinition import (
    EntityType,
    TestPlatform,
    TestDefinition,
)
from metadata.generated.schema.type.basic import FullyQualifiedEntityName
from metadata.ingestion.ometa.ometa_api import OpenMetadata

# ===== 配置 =====
OM_SERVER = "http://localhost:8585/api"
OM_TOKEN =
↵ "eyJraWQiOiJHYjM4OWEtOWY3Ni1nZGpzLWE5MmotMDI0MmJrOTQzNTYiLCJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9..."

```

```

# 初始化 OpenMetadata 客户端
metadata = OpenMetadata(
    host_port=OM_SERVER,
    token=OM_TOKEN,
)

# 验证连接
print(f"Server version: {metadata.client.get('/version')}")

def create_test_case(
    test_name: str,
    entity_fqn: str,
    test_definition_fqn: str,
    parameter_values: list = None
):
    """ 创建单个测试用例 """
    try:
        test_case = TestCase(
            name=test_name,
            entityLink=f"<{entity_fqn}>",
            entityFQN=entity_fqn,
            testDefinition=TestDefinition(
                fullyQualified_name=test_definition_fqn
            ),
            parameterValues=parameter_values or [],
        )
        created = metadata.create_or_update(test_case)
        print(f"$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/2713.png}}}}}$ Created test: {test_name}")
        return created
    except Exception as e:
        print(f"$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/2717.png}}}}}$ Failed to create test {test_name}: {e}")
        return None

def bulk_create_tests(yaml_file: str):
    """ 从 YAML 文件批量创建测试 """
    with open(yaml_file, 'r') as f:
        config = yaml.safe_load(f)

    test_cases = config.get('testCases', [])
    success_count = 0

    for tc in test_cases:
        parameter_values = []
        for pv in tc.get('parameterValues', []):
            parameter_values.append({
                "name": pv['name'],
                "value": pv['value']
            })

```

```

        result = create_test_case(
            test_name=tc['name'],
            entity_fqn=tc['entityFQN'],
            test_definition_fqn=tc['testDefinition'],
            parameter_values=parameter_values
        )
    if result:
        success_count += 1

print(f"\nSummary: {success_count}/{len(test_cases)} tests created successfully")

# 执行批量创建
if __name__ == "__main__":
    bulk_create_tests("material_quality_tests.yaml")

```

4.6 质量测试告警配置

```

# Step 1: 创建告警目标 (Slack 示例)
# 进入 OpenMetadata UI
# Settings > Alerts & Notifications > Add Alert Destination

cat > slack-alert-destination.json << 'EOF'
{
  "name": "material-quality-alerts",
  "displayName": " 物料质量告警",
  "description": " 物料数据质量测试失败时通知治理团队",
  "type": "slack",
  "config": {
    "webhookUrl": "https://hooks.slack.com/services/T00000000/B00000000/XXXXXXXXXXXXXXXXXXXX",
    "channel": "#data-governance-alerts"
  }
}
EOF

# Step 2: 创建告警规则
cat > quality-alert-rule.json << 'EOF'
{
  "name": "material-test-failure-alert",
  "displayName": " 物料测试失败告警",
  "description": " 当物料相关的数据质量测试失败时发送告警",
  "triggerConfig": {
    "type": "dataQuality",
    "fields": ["testCaseStatus"],
    "filters": [
      {
        "condition": "or",
        "rules": [
          {
            "name": "testCaseStatus",
            "operator": "eq",

```

```

        "value": "Failed"
      },
      {
        "name": "testCaseStatus",
        "operator": "eq",
        "value": "Aborted"
      }
    ]
  }
]
},
"destinationConfig": [
  {
    "destination": {
      "id": "material-quality-alerts",
      "type": "slack"
    },
    "sendToAdmins": true,
    "sendToOwners": true
  }
],
"enabled": true
}
EOF

# Step 3: 验证告警
# 在 Slack #data-governance-alerts 频道中应该收到:
# 🚨 Data Quality Test Failed
# Test: MARA_MATNR_NotNull
# Table: SAP-HANA-PROD.ERP.MARA
# Failed At: 2026-05-01 02:15:32
# Details: 3 rows have null values in column MATNR

```

第五部分：Glossary（术语表）构建

5.1 物料分类体系术语表

```

# Step 1: 创建 Glossary
curl -X POST http://localhost:8585/api/v1/glossaries \
  -H "Content-Type: application/json" \
  -u admin:admin \
  -d '{
    "name": "物料分类体系",
    "displayName": "物料分类体系",
    "description": "离散制造业物料主数据分类标准，基于 GB/T 7635 国家标准并适配企业实际需求",
    "mutuallyExclusive": true,    // 一个物料只能属于一个分类
    "owner": {
      "id": "data-steward-team",
      "type": "team"
    },
  },

```

```
"reviewers": [  
  {"id": "procurement-lead", "type": "user"},  
  {"id": "production-lead", "type": "user"},  
  {"id": "quality-lead", "type": "user"}  
],  
"tags": [" 物料", " 分类", " 主数据"]  
}'
```

Step 2: 创建一级分类 (大类)

2.1 原材料

```
curl -X POST http://localhost:8585/api/v1/glossaryTerms \  
-H "Content-Type: application/json" \  
-u admin:admin \  
-d '{  
  "name": " 原材料",  
  "displayName": " 原材料",  
  "description": " 直接用于产品生产加工的物料，经加工后构成产品实体",  
  "glossary": " 物料分类体系",  
  "synonyms": [" 原料", "Raw Material"],  
  "acronym": "RM",  
  "tags": ["P0", " 采购重点"],  
  "status": "Approved",  
  "owner": {"id": "procurement-lead", "type": "user"}  
}'
```

2.2 半成品

```
curl -X POST http://localhost:8585/api/v1/glossaryTerms \  
-H "Content-Type: application/json" \  
-u admin:admin \  
-d '{  
  "name": " 半成品",  
  "displayName": " 半成品",  
  "description": " 经过部分加工但尚未完成全部生产工序的中间产品",  
  "glossary": " 物料分类体系",  
  "synonyms": [" 在制品", "WIP", "Semi-finished"],  
  "acronym": "SF",  
  "tags": ["P1", " 生产管理"]  
}'
```

2.3 成品

```
curl -X POST http://localhost:8585/api/v1/glossaryTerms \  
-H "Content-Type: application/json" \  
-u admin:admin \  
-d '{  
  "name": " 成品",  
  "displayName": " 成品",  
  "description": " 已完成全部生产工序并经检验合格的产品",  
  "glossary": " 物料分类体系",  
  "synonyms": [" 产成品", "Finished Goods"],  
  "acronym": "FG",  
  "tags": ["P0", " 销售"]  
}'
```



```
}'
```

2.4 辅料

```
curl -X POST http://localhost:8585/api/v1/glossaryTerms \
-H "Content-Type: application/json" \
-u admin:admin \
-d '{
  "name": " 辅料",
  "displayName": " 辅料",
  "description": " 辅助生产的物料，不构成产品实体但在生产过程中消耗",
  "glossary": " 物料分类体系",
  "synonyms": [" 辅助材料", " 间接材料", "Auxiliary Material"],
  "acronym": "AM",
  "tags": ["P2"]
}'
```

2.5 备品备件

```
curl -X POST http://localhost:8585/api/v1/glossaryTerms \
-H "Content-Type: application/json" \
-u admin:admin \
-d '{
  "name": " 备品备件",
  "displayName": " 备品备件",
  "description": " 用于设备维修和维护的替换零部件",
  "glossary": " 物料分类体系",
  "synonyms": [" 备件", "Spare Parts"],
  "acronym": "SP",
  "tags": ["P2", " 设备管理"]
}'
```

Step 3: 创建二级分类（以原材料为例）

3.1 金属材料

```
curl -X POST http://localhost:8585/api/v1/glossaryTerms \
-H "Content-Type: application/json" \
-u admin:admin \
-d '{
  "name": " 金属材料",
  "displayName": " 金属材料",
  "description": " 以金属元素为主要成分的原材料",
  "glossary": " 物料分类体系",
  "parent": " 原材料",
  "tags": ["P0"]
}'
```

3.2 非金属材料

```
curl -X POST http://localhost:8585/api/v1/glossaryTerms \
-H "Content-Type: application/json" \
-u admin:admin \
-d '{
  "name": " 非金属材料",
  "displayName": " 非金属材料",
```

```

    "description": " 不以金属元素为主要成分的原材料",
    "glossary": " 物料分类体系",
    "parent": " 原材料",
    "tags": ["P1"]
  }'

# Step 4: 创建三级分类（以金属材料为例）
# 4.1 钢材
curl -X POST http://localhost:8585/api/v1/glossaryTerms \
  -H "Content-Type: application/json" \
  -u admin:admin \
  -d '{
    "name": " 钢材",
    "displayName": " 钢材",
    "description": " 铁碳合金材料，包括碳钢、合金钢、不锈钢等",
    "glossary": " 物料分类体系",
    "parent": " 金属材料",
    "synonyms": ["Steel", " 碳钢", " 不锈钢"],
    "tags": ["P0", " 大宗采购"]
  }'

# 4.2 铝材
curl -X POST http://localhost:8585/api/v1/glossaryTerms \
  -H "Content-Type: application/json" \
  -u admin:admin \
  -d '{
    "name": " 铝材",
    "displayName": " 铝材",
    "description": " 铝合金材料，包括铝板、铝棒、铝型材等",
    "glossary": " 物料分类体系",
    "parent": " 金属材料",
    "synonyms": [" 铝合金", "Aluminum", "Aluminium"],
    "tags": ["P1"]
  }'

```

5.2 将 Glossary 与数据库表关联

```

# 将 SAP MARA 表的 MTART（物料类型）字段关联到 Glossary
# 这样当用户浏览该字段时，可以看到物料类型的标准定义

# 方式 1: 通过 API 添加标签
curl -X PUT http://localhost:8585/api/v1/tables/SAP-HANA-PROD.ERP.MARA/columns/MTART/tags \
  -H "Content-Type: application/json" \
  -u admin:admin \
  -d '{
    "tags": [
      {
        "tagFQN": " 物料分类体系. 原材料",
        "description": " 该字段值对应物料分类体系中的分类",
        "source": "Classification",
        "labelType": "Manual",

```

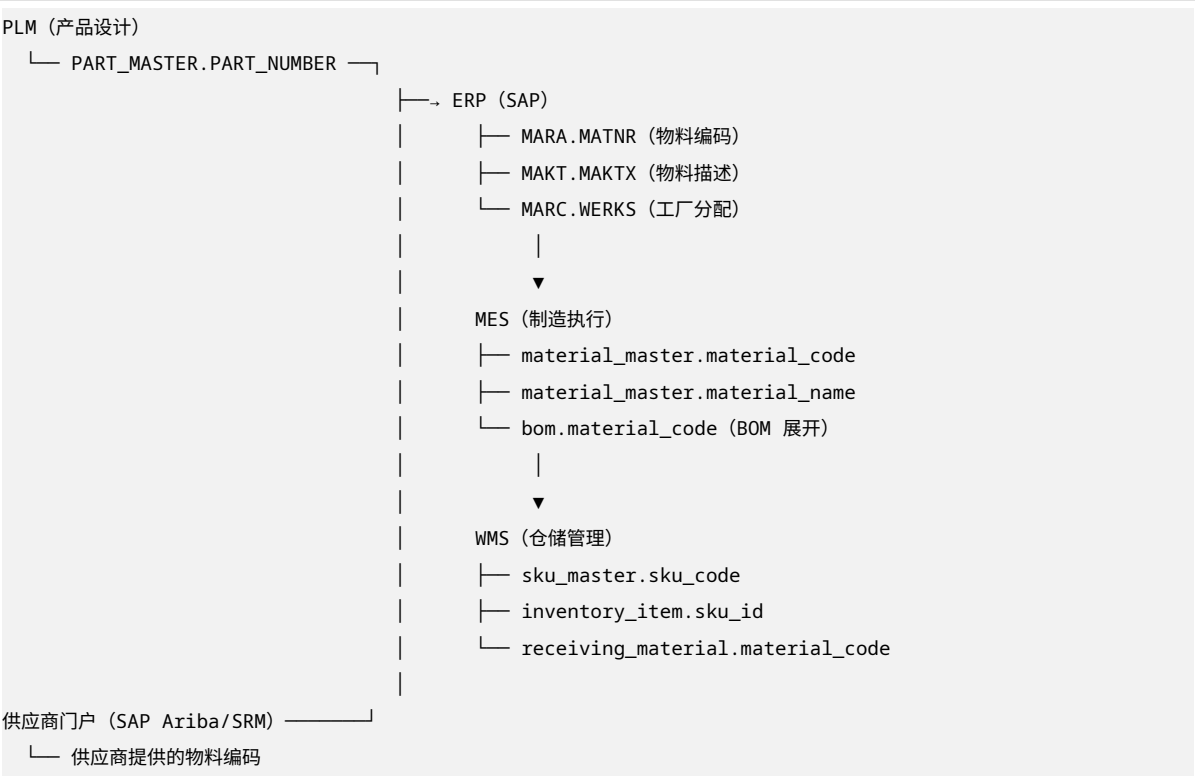
```
        "state": "Confirmed"
    }
  ]
}'

# 方式 2: 使用 OpenMetadata 的 Auto-Classification
# 在 Data Profiler 配置中启用 autoClassification
# 系统会自动扫描字段内容并建议合适的 Glossary 标签
```

第六部分：列级血缘追踪建设

6.1 血缘追踪方案设计

制造业物料数据的典型流转路径：



6.2 自动血缘采集

OpenMetadata 通过以下方式自动采集血缘：

方式 1：数据库查询日志分析（Query Log）

```
# 配置 SAP HANA 查询日志采集
cat > sap-usage-lineage.json << 'EOF'
{
  "name": "sap-hana-usage-lineage",
  "serviceName": "SAP-HANA-PROD",
  "pipelineType": "usage",
  "sourceConfig": {
    "config": {
```

```

    "type": "DatabaseUsage",
    "queryLogDuration": 1,           // 分析最近 1 天的查询日志
    "resultLimit": 10000,
    "queryFilterPattern": {
        "includes": ["SELECT", "INSERT", "UPDATE"] // 只分析 DML 语句
    },
    "schemaFilterPattern": {
        "includes": ["ERP"]
    }
  },
  "scheduleInterval": {
    "type": "custom",
    "cron": "0 5 * * *" // 每天凌晨 5 点分析前一天的查询日志
  }
}
EOF

# 注意：需要在数据库端开启查询日志
# SAP HANA：需要 AUDIT 权限和查询历史表 (M_SQL_PLAN_CACHE, M_EXECUTED_STATEMENTS)
# MySQL：需要开启 general_log 或 slow_query_log
# PostgreSQL：需要 pg_stat_statements 扩展

```

方式 2：DBT 集成（推荐，最准确）

如果企业使用 dbt 进行数据转换，OpenMetadata 可以直接解析 dbt 的 manifest.json：

```

# 配置 dbt 血缘采集
cat > dbt-lineage.json << 'EOF'
{
  "name": "dbt-material-lineage",
  "serviceName": "dbt-project",
  "pipelineType": "metadata",
  "sourceConfig": {
    "config": {
      "type": "Dbt",
      "dbtConfigSource": {
        "dbtManifestFilePath": "/opt/dbt/target/manifest.json",
        "dbtCatalogFilePath": "/opt/dbt/target/catalog.json",
        "dbtRunResultsFilePath": "/opt/dbt/target/run_results.json",
        "dbtSourcesPath": "/opt/dbt/models/staging"
      },
      "databaseServiceName": "SAP-HANA-PROD"
    }
  }
}
EOF

```

方式 3：手动血缘编辑

对于无法自动采集的血缘关系，可以通过 UI 手动添加：

操作路径：

1. 导航到 Explore > Tables > SAP-HANA-PROD > ERP > MARA
2. 切换到 "Lineage" Tab
3. 点击 "Edit"
4. 拖拽添加上下游节点
5. 连接具体的列级关系 (如 MARA.MATNR → MES.material_master.material_code)
6. 点击 "Save"

6.3 血缘的应用场景

场景 1：变更影响分析

当计划修改 SAP MARA 表的 MATNR（物料编码）字段长度时：

- 操作：
1. 打开 MARA.MATNR 的 Lineage 页面
 2. 查看 "Downstream"（下游）依赖
 3. 系统显示：
 - MES.material_master.material_code（直接依赖）
 - WMS.sku_master.sku_code（间接依赖）
 - BI.material_inventory_report（报表依赖）
 - 共影响 12 个下游系统/表
 4. 导出影响分析报告，发送给相关团队评估

场景 2：数据问题溯源

当发现 MES 系统中某物料名称错误时：

- 操作：
1. 打开 MES.material_master 的 Lineage 页面
 2. 查看 "Upstream"（上游）来源
 3. 系统显示数据来源路径：
SAP.MAKT.MAKTX → ERP_SYNC_VIEW.material_desc → MES.material_master.material_name
 4. 追溯到源头是 SAP MAKT 表的描述字段
 5. 在 SAP 端修正后，通过血缘确认下游已全部同步

第七部分：KPI 看板与治理运营

7.1 数据质量 KPI 配置

```
# 在 OpenMetadata 中配置 KPI
# 路径: Observability > KPIs > Add KPI

cat > material-quality-kpis.json << 'EOF'
[
  {
    "name": "material-description-completeness",
    "displayName": "  物料描述完整率",
    "metricType": "completedDescriptionFraction",
    "targetDefinition": {
      "type": "completedDescriptionFraction",
      "value": 0.95      // 目标: 95% 的物料有描述
    }
  }
]
```

```
    },
    "startDate": "2026-05-01",
    "endDate": "2026-12-31"
  },
  {
    "name": "material-code-uniqueness",
    "displayName": " 物料编码唯一率",
    "metricType": "custom",
    "targetDefinition": {
      "type": "custom",
      "sql": "SELECT COUNT(DISTINCT MATNR) / COUNT(*) FROM MARA",
      "value": 1.00      // 目标: 100% 唯一
    }
  },
  {
    "name": "unit-standardization-rate",
    "displayName": " 计量单位标准化率",
    "metricType": "custom",
    "targetDefinition": {
      "type": "custom",
      "sql": "SELECT SUM(CASE WHEN MEINS IN ('KG','PCS','M','L') THEN 1 ELSE 0 END) / COUNT(*) FROM
      ↵ MARA",
      "value": 0.98      // 目标: 98% 标准化
    }
  }
]
EOF
```

7.2 数据洞察报告 (Data Insights)

OpenMetadata 自动生成以下报告：

报告类型	内容	用途
App Analytics	平台使用活跃度	评估治理团队参与度
Data Assets	表/列/服务数量趋势	评估元数据采集覆盖度
Data Quality	测试用例数量、通过率趋势	评估数据质量改善效果
KPIs	KPI 目标达成进度	治理项目进度监控

治理周报自动生成：

```
#!/usr/bin/env python3
""" 数据治理周报自动生成脚本 """

from metadata.ingestion.ometa.ometa_api import OpenMetadata
from datetime import datetime, timedelta

metadata = OpenMetadata(
    host_port="http://localhost:8585/api",
    token="YOUR_JWT_TOKEN"
)
```

```

# 获取本周数据质量统计
end_date = datetime.now()
start_date = end_date - timedelta(days=7)

# 获取测试用例执行结果
test_cases = metadata.list_all_entities(TestCase)

success = sum(1 for tc in test_cases if tc.testCaseResult and tc.testCaseResult.testCaseStatus ==
    ↳ "Success")
failed = sum(1 for tc in test_cases if tc.testCaseResult and tc.testCaseResult.testCaseStatus ==
    ↳ "Failed")
total = success + failed

report = f"""
┌────────────────────────────────────────────────────────────────────────────────┐
│          物料主数据治理周报 ( {start_date.date()} ~ {end_date.date()} )          │
└────────────────────────────────────────────────────────────────────────────────┘
┌────────────────────────────────────────────────────────────────────────────────┐
│ 数据质量测试概况                                                                │
├────────────────────────────────────────────────────────────────────────────────┤
│ 总测试用例: {total:>5}                                                            │
│ 通过:        {success:>5}  ( {success/total*100:.1f}% )                          │
│ 失败:        {failed:>5}  ( {failed/total*100:.1f}% )                          │
│                                                                                   │
│ 本周新增测试: X 个                                                                │
│ 修复的问题:   Y 个                                                                │
│                                                                                   │
│ TOP 3 问题表:                                                                    │
│ 1. MARA - 编码格式不规范 (12 条)                                                │
│ 2. MAKT - 描述缺失 (8 条)                                                       │
│ 3. T_BD_MATERIAL - 单位不标准 (5 条)                                            │
└────────────────────────────────────────────────────────────────────────────────┘
"""

print(report)

# 发送到 Slack/邮件
# ...

```

第八部分：高级功能与集成

8.1 Data Quality as Code

从 1.12 版本开始，OpenMetadata 支持通过 YAML 文件管理数据质量测试，实现版本控制和 CI/CD 集成：

```

# om_tests.yaml
# 存放在 Git 仓库中，通过 CI/CD 自动部署

source: "SAP-HANA-PROD.ERP"
testCases:

```

```

- name: MARA_MATNR_NotNull
  entityFQN: SAP-HANA-PROD.ERP.MARA
  columnName: MATNR
  testDefinition: columnValuesToBeNotNull
  scheduleInterval: "0 2 * * *" # 每天凌晨 2 点

- name: MARA_MTART_ValidSet
  entityFQN: SAP-HANA-PROD.ERP.MARA
  columnName: MTART
  testDefinition: columnValuesToBeInSet
  parameterValues:
    - name: allowedValues
      value: "['ROH','HALB','FERT','HAWA','HIBE','ERSA']"
  scheduleInterval: "0 2 * * *"

# CI/CD Pipeline (GitHub Actions 示例)
# .github/workflows/deploy-quality-tests.yml
"""
name: Deploy Data Quality Tests
on:
  push:
    branches: [main]
    paths: ['om_tests.yaml']

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Deploy to OpenMetadata
        run: |
          pip install openmetadata-ingestion[sdk]
          python deploy_tests.py --config om_tests.yaml --env production
"""

```

8.2 GitHub Metadata Sink（元数据版本控制）

将 OpenMetadata 的元数据变更自动同步到 Git 仓库：

```

# 配置 GitHub Sink
cat > github-sink-config.json << 'EOF'
{
  "type": "github",
  "config": {
    "repositoryName": "company/metadata-repo",
    "branch": "main",
    "token": "${GITHUB_TOKEN}",
    "filePath": "metadata/{entityType}/{entityName}.json"
  }
}
EOF

```



```
# 效果：每次在 OpenMetadata 中修改表描述、添加标签、更新测试
# 都会自动提交到 Git 仓库，形成审计追踪
```

8.3 MCP 集成 (AI 助手)

OpenMetadata 1.12 支持 MCP (Model Context Protocol), 可与 AI 助手集成:

```
# 使用 Claude/Cursor 等 AI 助手查询 OpenMetadata
# 用户可以用自然语言提问:

"Show me all material tables with data quality issues"
"What is the lineage of MARA.MATNR?"
"Which tables have duplicate material codes?"
"Compare data quality between SAP and Kingdee systems"

# AI 助手会自动调用 OpenMetadata API 获取答案
```

第九部分：常见问题与解决方案

FAQ 1: SAP 数据库连接失败

症状: 配置 SAP HANA 连接器后, 采 Pipeline 报连接超时。

排查步骤:

```
# 1. 检查网络连通性
telnet sap-hana.company.internal 30015

# 2. 检查防火墙规则
# 需要开放 30013-30018 端口 (HANA 默认端口范围)

# 3. 验证数据库用户权限
# 在 HANA Studio 中执行:
SELECT * FROM MARA LIMIT 1;
-- 确认 OPENMETADATA 用户有 SELECT 权限

# 4. 检查 OpenMetadata 日志
docker logs openmetadata_server | grep -i "sap\|hana\|connection"
```

FAQ 2: 数据质量测试执行慢

症状: 质量测试执行时间超过 1 小时。

优化方案:

```
# 1. 减少采样比例
# Profiler 配置中: profileSample: 10 (10% 采样)

# 2. 增加并发线程
# Profiler 配置中: threadCount: 10
```

```
# 3. 为经常测试的大表创建索引
# SAP HANA:
CREATE INDEX idx_mara_matnr ON MARA(MATNR);

# 4. 分区测试（避免在业务高峰期执行）
# scheduleInterval: "0 2 * * *" # 凌晨 2 点
```

FAQ 3: 中文物料描述显示乱码

解决方案:

```
# 确保数据库连接字符集正确
# MySQL 连接参数:
"connectionOptions": {
  "useUnicode": "true",
  "characterEncoding": "UTF-8",
  "serverTimezone": "Asia/Shanghai"
}

# SAP HANA 连接参数:
"connectionOptions": {
  "CHARSET": "UTF-8"
}
```

FAQ 4: Docker 部署内存不足

症状: OpenMetadata 容器频繁 OOMKilled。

解决方案:

```
# docker-compose-postgres.yml 中增加内存限制
services:
  openmetadata_server:
    deploy:
      resources:
        limits:
          memory: 4G
        reservations:
          memory: 2G

  elasticsearch:
    deploy:
      resources:
        limits:
          memory: 2G
        reservations:
          memory: 1G

# 如果内存仍然不足, 可以禁用 Elasticsearch (使用 database 搜索模式)
# env-postgres 中: SEARCH_TYPE=database
```

第十部分：运维与持续优化

10.1 日常运维 checklist

```
# 每日检查
□ 查看数据质量测试执行结果
  └─ 路径: Observability > Data Quality
□ 处理失败的测试用例
□ 查看告警通知 (Slack/邮件)

# 每周检查
□ 检查元数据采 Pipeline 执行状态
□ 查看 KPI 达成进度
□ 审查 Glossary 变更申请
□ 生成数据治理周报

# 每月检查
□ 评估数据质量改善趋势
□ 优化质量测试规则
□ 审查和更新分类体系
□ 进行系统备份

# 每季度检查
□ 全面评估治理成效
□ 扩展治理范围 (新增系统/表)
□ 升级 OpenMetadata 版本
□ 培训新成员
```

10.2 备份策略

```
# 数据库备份 (PostgreSQL)
#!/bin/bash
BACKUP_DIR="/backup/openmetadata"
DATE=$(date +%Y%m%d_%H%M%S)

# 备份 OpenMetadata 元数据数据库
docker exec postgres pg_dump -U postgres openmetadata_db > \
  $BACKUP_DIR/om_metadata_$DATE.sql

# 备份 Elasticsearch 索引
docker exec elasticsearch curl -X POST \
  "localhost:9200/_snapshot/backup_repo/snapshot_$DATE"

# 保留最近 30 天的备份
find $BACKUP_DIR -name "*.sql" -mtime +30 -delete
```

10.3 版本升级指南

```
# 1. 备份当前环境
docker compose -f docker-compose-postgres.yml exec postgres \
  pg_dump -U postgres openmetadata_db > om_backup_pre_upgrade.sql
```

```
# 2. 拉取新版本镜像
export OM_VERSION=1.13.0 # 目标版本
sed -i "s/image: openmetadata\/server:.*\/image: openmetadata\/server:${OM_VERSION}\/" \
  docker-compose-postgres.yml

# 3. 执行数据库迁移
docker compose -f docker-compose-postgres.yml run --rm openmetadata_server \
  bash -c ".\/bootstrap\/openmetadata-ops.sh migrate"

# 4. 启动新版本
docker compose -f docker-compose-postgres.yml up -d

# 5. 验证升级
curl http://localhost:8585/api/v1/health-check
```

附录：参考资源

资源	链接
OpenMetadata 官网	https://open-metadata.org
官方文档	https://docs.open-metadata.org
GitHub 仓库	https://github.com/open-metadata/OpenMetadata
社区 Slack	https://slack.open-metadata.org
博客	https://blog.open-metadata.org
快速开始指南	https://docs.open-metadata.org/v1.12.x/quick-start
连接器列表	https://docs.open-metadata.org/v1.12.x/connectors
数据质量文档	https://docs.open-metadata.org/v1.12.x/how-to-guides/data-quality-observability
Test Library 文档	https://docs.open-metadata.org/v1.12.x/how-to-guides/data-quality-observability/quality/test-library
Python SDK	https://docs.open-metadata.org/v1.12.x/sdk/python
API 参考	https://docs.open-metadata.org/v1.12.x/api-reference
版本发布说明	https://docs.open-metadata.org/v1.12.x/releases
安全指南	https://docs.open-metadata.org/v1.12.x/deployment/security
Kubernetes 部署	https://docs.open-metadata.org/v1.12.x/deployment/kubernetes

文档版本: v1.0 最后更新: 2026-05-02 适用 OpenMetadata 版本: 1.12.x